

LABORATORY EXERCISE NO. 6

CRUD with Authentication Using React/Vue.js and LavaLust API

Frontend: React.js or Vue.js

Backend: LavaLust API

Database: Aiven MySQL

Deployment: Render

Instructions

Create a **Product Management System** using **React.js or Vue.js** for the frontend and **LavaLust API** for the backend.

1. Database

Create a table named:

products

Column	Data Type
id	INT, Primary Key, Auto Increment
product_name	VARCHAR(100)
description	TEXT
price	DECIMAL(10,2)
quantity	INT
created_at	TIMESTAMP

Use **Aiven MySQL** as your database.

2. LavaLust API

Create a LavaLust API that will handle the product data.

Your API must support:

- **GET** - display products
- **POST** - add a product
- **PUT/PATCH** - update a product

- **DELETE** – delete a product

Use the **LavaLust API class** for your API responses.

3. Authentication

Implement authentication for the application.

You may use:

- LavaLust authentication/middleware
- Token-based authentication
- Session-based authentication
- Another appropriate authentication method

Only authenticated users should be allowed to perform product CRUD operations.

4. React.js or Vue.js

Create a frontend application that can:

- Login
- Display products
- Add a product
- Edit a product
- Delete a product
- Logout

The frontend must communicate with the LavaLust API using HTTP requests.

5. Deployment

Deploy your **LavaLust API to Render** and connect it to your **Aiven MySQL database**.

Your React/Vue application should communicate with the deployed LavaLust API.

6. Required Output

Your final application should demonstrate:

Login

↓

Product List

↓

Add / Edit / Delete

7. Submission

Submit the following:

1. GitHub repository of the LavaLust API
2. GitHub repository of the React/Vue application
3. Render API URL
4. Frontend URL
5. Screenshots of:
 - o Login
 - o Product list
 - o Add product
 - o Edit product
 - o Delete product
 - o Aiven database
6. Working demonstration of the application

Important

- Do not expose database passwords in GitHub.
- Use environment variables for sensitive credentials.
- The React/Vue application must **not connect directly to Aiven MySQL**.

- All database operations must pass through the **LavaLust API**.
- CRUD operations must be protected by authentication.
- You may design your own user interface.

Goal: Develop a simple full-stack CRUD application using **React.js/Vue.js + LavaLust API + Aiven MySQL + Render**.

References:

LavaLust API Library

<https://lavalust.netlify.app/docs/libraries/api.html>

Tutorials:

<https://www.geeksforgeeks.org/reactjs/how-to-fetch-data-from-an-api-in-reactjs/>

<https://www.geeksforgeeks.org/reactjs/axios-in-react-a-guide-for-beginners/>

API Tester:

<https://api-tester.marasigan.dev/>

LABORATORY ACTIVITY

Database Migration Using LavaLust

Objective

At the end of this laboratory activity, students should be able to:

1. Use the **LavaLust CLI** to manage a project.
2. Create a controller using the LavaLust CLI.
3. Define routes for database migration operations.
4. Create a custom CLI command.
5. Execute database migrations.
6. Verify the tables created by the migration system.

Part I — Start the LavaLust Project

Open your LavaLust project in **Visual Studio Code** and open the terminal.

Start the development server:

```
php lava serve
```

Verify that the application is running successfully.

Part II — Create the Migration Controller

Use the LavaLust CLI to generate a controller:

```
php lava make:controller MigrationController
```

Open the generated controller and replace its contents with the following:

```
<?php
```

```
defined('PREVENT_DIRECT_ACCESS') OR exit('No direct script access allowed');
```

```
class MigrationController extends Controller
```

```
{
```

```
    public function __construct()
```

```
    {
```

```
        parent::__construct();
```

```
        $this->call->library('migration');
```

```
    }
```

```
    public function create_migration($migration_class)
```

```
    {
```

```
        $this->migration->create_migration($migration_class);
```

```
    }
```

```

public function migrate()
{
    $this->migration->migrate();
}

public function rollback()
{
    $this->migration->rollback();
}

public function rollback_all()
{
    $this->migration->rollback_all();
}

public function refresh()
{
    $this->migration->refresh();
}

public function status()
{
    $this->migration->status();
}
}

```

The controller provides access to the migration library through HTTP routes.

Part III — Configure the Routes

Open:

app/config/routes.php

Add the following routes:

```

// Migration Routes

$route->get('create-migration/{migration_class}',
'MigrationController::create_migration');

$route->get('migrate', 'MigrationController::migrate');
$route->get('rollback', 'MigrationController::rollback');
$route->get('rollback-all', 'MigrationController::rollback_all');
$route->get('refresh', 'MigrationController::refresh');
$route->get('status', 'MigrationController::status');

```

These routes provide access to the migration operations.

Migration Routes

Route	Function
/create-migration/{name}	Create a migration file
/migrate	Execute pending migrations
/rollback	Roll back the latest migration batch
/rollback-all	Roll back all migrations
/refresh	Roll back and re-run migrations
/status	Display migration status

Part IV — Create a Custom CLI Command

Instead of executing migration operations only through browser routes, create a custom Laravel CLI command.

Run:

```
php laravel make:command Migration
```

Open the generated command file inside:

```
app/commands/Migration.php
```

Paste the following:

```
<?php

class Migration
{
    public static $command = 'migration';

    public static $description = 'Run database migrations';

    public static $arguments = [
        '[action]' => 'Action: run, create-migration, rollback, rollback-all,
refresh, status',
        '[name]'   => 'Migration class name for create-migration',
    ];

    protected static $route_map = [
        'run'           => 'migrate',
        'create-migration' => 'create-migration',
        'rollback'       => 'rollback',
        'rollback-all'  => 'rollback-all',
        'refresh'        => 'refresh',
        'status'         => 'status',
    ];
}
```

```

public function handle($action = null, array $flags = [], $name = null)
{
    $action = $action ?? 'run';

    if (!isset(static::$route_map[$action])) {
        echo danger("Unknown migration action: \"{$action}\"");
        echo "Available actions: "
            . implode(', ', array_keys(static::$route_map))
            . PHP_EOL;
        exit(1);
    }

    if ($action === 'create-migration') {

        if (!$name) {
            echo danger("Migration name is required.");
            echo "Example: php lava migration create-migration
create_users_table"
                . PHP_EOL;
            exit(1);
        }

        $route = 'create-migration/' . $name;

    } else {

        $route = static::$route_map[$action];
    }

    $index = PUBLIC_DIR . 'index.php';

    if (!file_exists($index)) {
        echo danger("index.php not found at: {$index}");
        exit(1);
    }

    $command = sprintf(
        'php %s %s',
        escapeshellarg($index),
        escapeshellarg($route)
    );

    passthru($command);
}
}

```

Part V — Execute the Migration

Run the migration command:

```
php lava migration run
```

The command should execute the pending migration files.

The migration system will create the required database tables.

For the API authentication system, verify that the following tables are available:

users

refresh_tokens

The users table stores user account information, while refresh_tokens stores refresh-token data used by the API authentication system.

Part VI — Verify the Database

Open your database management tool, such as:

- MySQL Workbench
- phpMyAdmin
- DBeaver
- Navicat

Select your LavaLust database and verify that the migration-generated tables exist.

Expected tables:

migrations

users

refresh_tokens

The migrations table is used by LavaLust to track which migrations have already been executed.

Part VII — Test Other Migration Commands

Test the following commands:

Check Migration Status

```
php lava migration status
```

Create a Migration

```
php lava migration create-migration create_products_table
```

Run Pending Migrations

```
php lava migration run
```

Roll Back the Latest Migration

php lava migration rollback

Roll Back All Migrations

php lava migration rollback-all

Refresh the Database

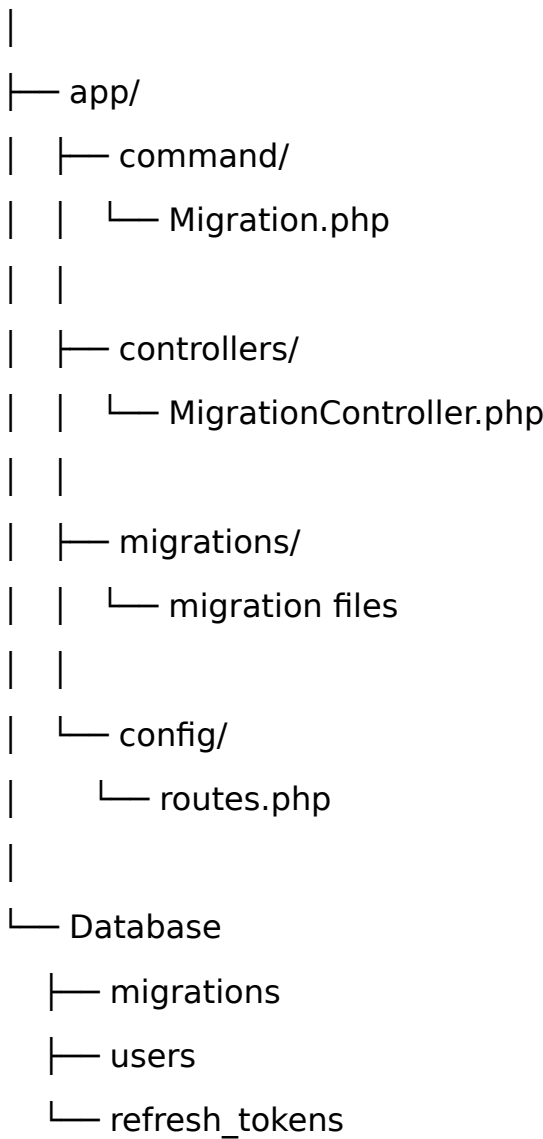
php lava migration refresh

Important: rollback-all and refresh can remove and recreate database structures. Use them only in your development database.

Expected Output

At the end of the activity, students should have:

LavaLust Project



Key Concepts Practiced

CLI → Controller → Route → Migration Library → Database

Students should understand that database migrations allow developers to manage database structure using **version-controlled PHP code instead of manually creating database tables**.